🎓 RentSphere: Teammate Setup Guide & Tomorrow's Viva Demo Walkthrough

MCA Object-Oriented Technology Laboratory Project · Goa University
Repository: https://github.com/mohammed2664/Mini-Project
Technology: Pure Java SE (JDK 17+/25) · Zero External JARs · Zero Database · Vanilla Web Frontend

---

📌 Table of Contents

1. Prerequisites & System Requirements
2. Step-by-Step Setup: Clone & Run (Under 2 Minutes)
3. Pre-Seeded User Accounts & Credentials
4. Foolproof 5-Minute Live Demo Script for Tomorrow
5. Professor Q&A Cheat Sheet (The 6 Big OOT Questions)
6. Emergency Troubleshooting & Quick Fixes

---

1. Prerequisites & System Requirements

Your system needs **only ONE dependency**: a standard Java Development Kit (JDK).

Component	Requirement	Notes
Operating System	Windows 10/11, macOS, or Linux	Works identically across all operating systems.
Java JDK	**JDK 17, 21, or 25**	No Maven, Gradle, or external libraries needed. If you don't have it, install Eclipse Temurin JDK 21.
Web Browser	Chrome, Edge, Firefox, or Brave	App runs locally at `http://localhost:8080/`.
Git (Optional)	Recommended	To clone updates directly from GitHub.

> [!NOTE]
> **No Database Server or Node.js Required:** RentSphere uses built-in Java SE `HttpServer` and atomic flat-file persistence. You do **not** need MySQL, PostgreSQL, Docker, NPM, or Python to run the application.

---

2. Step-by-Step Setup: Clone & Run (Under 2 Minutes)

Option A: Using Git (Recommended)

1. Open **Terminal** (Mac/Linux) or **PowerShell / Command Prompt** (Windows):

   ```
   git clone https://github.com/mohammed2664/Mini-Project.git
   cd Mini-Project
   ```

2. **Run on Windows (1-Click):**
   - Double-click `START_RENTSPHERE.bat` in the project folder.
   - *The script auto-detects your installed JDK, compiles the source files, starts the Java backend in a minimized console, and opens `http://localhost:8080/` in your browser.*

3. **Run on macOS / Linux (1-Click):**

   ```
   chmod +x start.sh run_tests.sh
   ./start.sh
   ```

Option B: Download as ZIP (No Git needed)

1. Go to the GitHub repository: github.com/mohammed2664/Mini-Project.
2. Click the green **Code** button and select **Download ZIP**.
3. Extract the ZIP to your computer (e.g., `Desktop\Mini-Project`).
4. Double-click `START_RENTSPHERE.bat` (Windows) or execute `./start.sh` (Mac/Linux).

Manual Command-Line Compilation (If you prefer running commands)

If you prefer to compile and run by hand in your terminal:

```
# 1. Compile all Java source files into the bin/ directory
javac -d bin -encoding UTF-8 @sources.txt

# 2. Start the RentSphere server on port 8080
java -cp bin rentsphere.api.MainServer 8080

# 3. Open your browser and visit:
http://localhost:8080/
```

Verifying the Automated Test Suite (163 / 163 Tests)

To show the examiners proof that all 17 test suites, concurrency checks, and WAL transactions pass:

- **Windows:** Double-click `RUN_TESTS.bat`.
- **Mac/Linux:** Run `./run_tests.sh`.
- **Manual Command:**

  ```
  java -cp bin tests.RentSphereVerificationTest
  ```

- **Expected Output:**

  ```
  =================================================================
    VERIFICATION SUMMARY: 163 / 163 TESTS PASSED (100% SUCCESS)
  =================================================================
  ```

---

3. Pre-Seeded User Accounts & Credentials

All user accounts use PBKDF2-HMAC-SHA256 password hashing. For convenience, **all demo accounts share the exact same password**:

> Universal Demo Password: `RentSphere@2026`

Persona	Username / Email	Role	Starting Wallet	Primary Purpose in Demo
Deepak Dev	`renter_deepak`	**RENTER**	₹14,180.00	Books gear, locks escrow collateral, retrieves OTP, returns gear.
Ketan Naik	`host_ketan`	**HOST**	₹8,200.00	Hosts Maruti Fronx & Ather Scooter; inspects returns & releases deposit.
Faaris Khan	`host_faaris`	**HOST**	₹5,500.00	Hosts Karcher Washer & Sony Camera; lists new equipment with photos.
Priya Sharma	`renter_priya`	**RENTER**	₹12,000.00	Used for demonstrating multi-renter concurrency & conflict isolation.
Prof. Almeida	`inspector_almeida`	**ADMIN**	₹50,000.00	Hub manager; inspects quarantine lockers, overrides bay telemetry, resolves disputes.

---

4. Foolproof 5-Minute Live Demo Script for Tomorrow

Follow this exact sequence in front of the professors. It is timed to take **under 5 minutes** and demonstrates every key OOT requirement:

```
Browse Gear (Polymorphism) ➔ Book on Product Page (Escrow) ➔ Unlock Locker Bay (State Pa
```

[0:00 – 0:30] Introduction & Opening Statement

- Double-click `START_RENTSPHERE.bat`.
- State clearly:
  > *"Good morning Sirs. Our project is RentSphere, a peer-to-peer equipment rental system with smart locker custody, designed for community gear sharing in Goa. It is built in pure Java SE with zero third-party dependencies, demonstrating core Object-Oriented Technology principles: a 5-tier inheritance hierarchy, the GoF State Pattern, a 4-gate fail-fast validation pipeline, concurrency control, and atomic flat-file persistence with a Write-Ahead Log."*

[0:30 – 1:15] Step 1: The Marketplace & Inheritance Hierarchy

- Click the category filter chips: **"Cars & mobility"**, **"Cameras & projectors"**, **"Tools & DIY"**, **"Audio & lighting"**.
- Explain:
  > *"Every item inherits from the abstract base class `RentableAsset`. Subclasses such as `CameraAndOpticsGear`, `PowerToolEquipment`, and `EventAndMobilityVehicle` polymorphically calculate daily rental tariffs and security deposits based on their category attributes—for example, 4K sensor resolutions or vehicle engine power."*

[1:15 – 2:00] Step 2: Dedicated Product Details & Escrow Reservation

- Find the **"Maruti Suzuki Fronx"** card and click **"Details ↗"** (or click the photo/title).
- Point to the dedicated product page:
  > *"This is our dedicated gear booking view. It displays full vehicle specifications (5 Seats, Automatic, Petrol, Insured), custody maintenance logs, and a live pricing calculator."*
- Select a 2-day duration (e.g., from tomorrow to +2 days).
- Point out the calculation breakdown:
  - Daily Tariff: `₹2,200.00 / day`
  - Total Tariff: `₹4,400.00`
  - Refundable Collateral: `₹6,000.00`
  - Total Escrow Commitment: `₹10,400.00`
- Click **"Confirm Booking"**.
- **Show the Green OTP Card:**
  > *"The system assigns locker Bay H2, locks the deposit in software escrow, transitions the asset state to `RESERVED_ESCROW`, and generates a secure 6-digit OTP code."*
- Note down or remember the 6-digit code shown on screen (e.g., `876909`).

[2:00 – 2:45] Step 3: Smart Locker Pickup & GoF State Pattern

- Click **"View in Pickup Hub ↗"** (or switch to the main tab and click the **"Pickup hub"** tab).
- Click on **Bay H2** in Row H:
  > *"Notice our spatial locker invariant: small items like cameras go into Rows A through G, but vehicles like the Fronx and Ather scooter are strictly restricted by our domain validation pipeline to Row H Ground Bays."*
- Click **"Verify OTP & Unlock Pickup"**.
- Enter the 6-digit OTP passcode.
- Explain:
  > *"When the OTP matches, the GoF State Pattern transitions the asset from `RESERVED_ESCROW` to `ACTIVE_LEASE`. If another user attempts to book or collect this asset now, the State pattern strictly rejects the transition."*

[2:45 – 3:30] Step 4: Return, Mandatory Quarantine & Host Settlement

- Click **"Return Gear to Hub Bay"**.
- Switch to the **"Returns & inspections"** tab.
- Explain:
  > *"Under our Bailment rules, when equipment is returned to the hub, it does NOT immediately become available to the public. It enters mandatory `INSPECTION_QUARANTINE`. The renter's deposit remains held in escrow until the host inspects the physical gear."*
- In the quarantine deck, show the item.
- Click **"Approve Clean Return & Release Escrow"**.
- Explain:
  > *"The host verifies clean condition. The software escrow engine releases the deposit back to Deepak's wallet, and the asset transitions back to `AVAILABLE`."*
- Point to the **Live Escrow & Audit Ledger** below:
  > *"Notice that every financial transfer and state transition was written to an append-only Write-Ahead Log in `data/audit_wal.txt`."*

[3:30 – 4:15] Step 5: Host Studio & Listing Onboarding with Real Photos

- Click the **"Host studio"** tab.
- Click **"+ List New Equipment"**.
- Show the modal:
  - Click one of the one-click gear presets (e.g., **"🚗 Fronx Car"** or **"📷 Sony A7 IV"**).
  - Point out that the image preview, category, and rates populate automatically.
  - Explain:
    > *"Hosts can upload custom image URLs or choose presets. The image metadata persists across server reboots via our pipe-delimited flat-file engine without breaking database schema invariants."*

[4:15 – 5:00] Step 6: Proof of Correctness (Automated Verification Suite)

- Minimize the browser, switch to the desktop/terminal, and run `RUN_TESTS.bat` (or `./run_tests.sh`).
- Let the professors watch the test suites execute live in console:
  - 5-Tier Inheritance tests
  - GoF State Machine tests
  - 4-Gate Fail-Fast validation pipeline
  - 8×8 Spatial matrix validation
  - Concurrency stress tests with simultaneous threads
  - Write-Ahead Log audit verification
- Point to the final summary banner:

  ```
  =================================================================
    VERIFICATION SUMMARY: 163 / 163 TESTS PASSED (100% SUCCESS)
  =================================================================
  ```

- End with:
  > *"Thank you Sirs. We are happy to take any questions regarding our Java domain models, concurrency handling, or architecture."*

---

5. Professor Q&A Cheat Sheet (The 6 Big OOT Questions)

Keep your answers short, calm, and grounded:

Q1: "Where is Inheritance and Polymorphism implemented in your code?"

> **Answer:** "*In our domain models under `rentsphere.model`. We have an abstract base class `RentableAsset` and specialized subclasses: `CameraAndOpticsGear`, `PowerToolEquipment`, `EventAudioAndLighting`, and `EventAndMobilityVehicle`. Each subclass overrides `calculateDailyRentalRate()` and `calculateSecurityDeposit()` polymorphically based on its specific attributes—for example, sensor megapixels for cameras or vehicle safety ratings.*"

Q2: "Why did you use the GoF State Pattern instead of boolean flags like `isReserved` or `isReturned`?"

> **Answer:** "*Using boolean flags leads to fragile, scattered if-else ladders that easily allow invalid transitions—like picking up an item before it is reserved. With the State Pattern in `AssetState.java`, each state (`AvailableState`, `ReservedEscrowState`, `ActiveLeaseState`, `InspectionQuarantineState`) is an independent class. Each state explicitly permits only legal operations and throws `IllegalStateException` on invalid actions, guaranteeing our mandatory quarantine invariant.*"

Q3: "How do you achieve Concurrency Control and Thread Safety without a SQL database?"

> **Answer:** "*In `EscrowVault.java` and `LockerGrid.java`, we use Java's `java.util.concurrent` locks—specifically `ReentrantLock` and synchronized critical sections around financial debits and state mutations. In addition, every state change is recorded sequentially to an append-only Write-Ahead Log (`audit_wal.txt`) protected by file locks. Our automated test suite includes a 4-way concurrent reservation test that proves only one thread succeeds while competing threads are cleanly rejected.*"

Q4: "What is your 4-Gate Fail-Fast Validation Pipeline?"

> **Answer:** "*Located in `RentalValidationPipeline.java`. Before any reservation can lock escrow capital or assign a locker bay, the request must pass through four sequential gates:*
>
> 1. *`KycIdentityGate`: Verifies user identity and prevents anti-self-rental (hosts cannot rent their own gear).*
> 2. *`AssetLifecycleGate`: Ensures the asset is in `AVAILABLE` state.*
> 3. *`EscrowSolvencyGate`: Checks that the renter's wallet has sufficient funds for both rental rent and security collateral.*
> 4. *`LockerMatrixGate`: Verifies spatial dimensions and allocates an appropriate locker bay.*"

Q5: "Why are vehicles restricted to Row H ground bays?"

> **Answer:** "*In `LockerGrid.java`, physical locker bays A through G are small, medium, and large lockers suited for cameras and tools. Vehicles like cars and scooters physically cannot fit into locker compartments. Our `LockerMatrixGate` checks `asset.requiresGroundBay()`, and if true, strictly mandates allocation only to Row H ground parking bays (H1 to H8).*"

Q6: What to say if a professor asks a bizarre, out-of-scope question:

> **The Truthful Bridge Phrase:**
> *"Sir, that specific aspect is outside the scope of our academic OOT prototype. In our current implementation, we focused on demonstrating pure Java SE object-oriented design patterns, thread-safe memory management, and deterministic file persistence. If we were deploying this to a commercial production cloud, that is definitely the next architectural enhancement we would implement."*

---

6. Emergency Troubleshooting & Quick Fixes

Issue 1: "Address already in use: bind (Port 8080)"

Cause: A previous Java server instance is still running in the background.
Fix (Windows): Open PowerShell and run:

```
$pids = Get-NetTCPConnection -LocalPort 8080 -State Listen -ErrorAction SilentlyContinue | Sel
if ($pids) { $pids | ForEach-Object { Stop-Process -Id $_ -Force } }
```

Then double-click `START_RENTSPHERE.bat` again.

Issue 2: "'javac' is not recognized as an internal or external command"

Cause: Java is installed, but its `bin` folder is not on your Windows `PATH`.
Fix:

1. Check if Java is in `C:\Program Files\Java\`.
2. Our updated `START_RENTSPHERE.bat` automatically scans `C:\Program Files\Java\jdk*` and uses it.
3. If you don't have JDK installed, download and install JDK 21 from Adoptium.

Issue 3: "Items are showing as Reserved from a previous practice run"

Cause: Your local ledger file has recorded previous test reservations.
Fix: Reset the ledger files to the clean repository baseline with one command:

```
git restore data/
```

Or restart the server via `START_RENTSPHERE.bat`.

All the best for the viva tomorrow! You have a robust, 100% verified pure Java SE system. Stay calm, speak clearly, and let the working code speak for itself. 🚀